

Croyez Immigration — Frontend Assessment

Project Overview

This project is a frontend implementation of the Croyez Immigration About page based on the provided design screenshot.

The application is built using Next.js 15 and Tailwind CSS v4 with a focus on:

- Responsive UI implementation
- Component-based architecture
- Performance optimization
- Accessibility and semantic HTML
- Clean and maintainable code structure

The About page is divided into reusable sections such as Banner, Services, Steps, Reviews, and Achievements for better scalability and maintainability.

Key Features

- Built with Next.js App Router
 - Styled using Tailwind CSS v4
 - Lazy loading implemented using `next/dynamic`
 - Reusable navigation and footer components
 - Responsive layout across screen sizes
 - Inline SVG usage without external icon libraries
 - Accessibility support using semantic elements and ARIA attributes
-

Setup & Run Instructions

1. Clone the Repository

- `git clone https://github.com/VishramR/Croyez_Assessment.git`

2. Open the Project in VS Code

- `cd <project-folder>`
- `code .`

3. Install Dependencies

- `npm install`

4. Run Development Server

- `npm run dev`

Open the browser and visit:

- `http://localhost:3000`

5. Production Build

- `npm run build`
- `npm start`

Folder Structure

- /public
 - └─ immigration.jpg
-
- /app
 - └─ layout.js
 - └─ globals.css
 - └─ page.js
-
- └─ /about
 - └─ page.js
 - └─ /sections
 - └─ Banner.jsx
 - └─ WhoWeAre.jsx
 - └─ WhyChoose.jsx
 - └─ Services.jsx
 - └─ Steps.jsx
 - └─ ClientReviews.jsx
 - └─ Achievements.jsx
-
- └─ /components
 - └─ Footer.jsx
 - └─ /nav
 - └─ NavHead.jsx
 - └─ Navbar.jsx
 - └─ NavLinks.jsx
 - └─ Topbar.jsx

Known Limitations

- The implementation is based solely on the provided screenshot reference.
 - Original design assets were unavailable, so similar placeholder/stock assets were used where necessary.
-

Additional Notes

- Banner and WhoWeAre are eagerly loaded for faster initial rendering.

- Remaining sections are dynamically imported for performance optimization.
- Custom utility classes are defined in `globals.css`.
- Semantic HTML and accessibility best practices are followed throughout the application.
-